Vision 기본 모델 Day 4

객체 탐지 — bbox · IoU · NMS · mAP · YOLO

HYUNDAI AI Insight Campus · 온디바이스 AI · Vision 기본 모델 4/5

2026. 11. 2 (월) 09:00–18:00 · 8교시 · Google Colab T4 GPU

bbox | IoU · NMS | mAP | Faster R-CNN · SSD | YOLOv8 | 영상 인원 카운트

" 무엇이 " → " 무엇이 어디에 "

강사 : 김정인 · 김구수

오늘의 학습 목표

수업이 끝나면 여러분은 다음을 할 수 있습니다

1 분류 → 탐지

Penn-Fudan 어노테이션을 파싱하고, sliding window 비용 (806창 · 약 4.8초/장) 으로 탐지기가 필요한 이유를 확인한다

2 bbox · COCO 포맷

xyxy · xywh · cxcywh 정규화를 서로 변환하고, Penn-Fudan 을 COCO JSON (annotations 423개) 으로 만든다

3 IoU · NMS

IoU 와 greedy NMS 를 직접 구현해 torchvision.ops 와 같은 결과를 내고, conf · IoU threshold 의 차이를 실험한다

4 Precision · Recall · mAP

TP · FP 매칭부터 PR 곡선 · AP · mAP@0.5:0.95 까지 직접 구현해 iou_nms_map.py 로 저장한다

5 2-stage · 1-stage · YOLO

Faster R-CNN 과 SSDLite 를 비교하고, YOLOv8 원시 출력 (1, 84, 8400) 을 직접 디코드해 ultralytics 결과와 맞춘다

6 영상 · 미니 프로젝트

영상 탐지 파이프라인을 만들고, 직접 만든 mAP 로 YOLOv8n · s 를 평가해 threshold 와 해상도를 고른다

시간표

8 교시 × 50 분 · 점심 12:00–13:00 · 오전 bbox → IoU · NMS, 오후 mAP → Faster R-CNN · SSD → YOLO → 영상 → 미니 프로젝트

교시	시간	주제	형태	애니메이션
1	09:00–09:50	Day 1~3 복습 퀴즈 · 분류기로 " 사람이 몇 명 ?"	퀴즈 + 강의	D4-1 분류 vs 탐지
2	10:00–10:50	탐지 문제 정의 — bbox 표현 · COCO 포맷	강의 + 실습	D4-2 bbox 좌표계
3	11:00–11:50	IoU · NMS	강의 + 실습	D4-3 IoU · NMS
4	13:00–13:50	평가 지표 — Precision · Recall · AP · mAP	강의 + 실습	D4-4 PR 곡선과 AP
5	14:00–14:50	2-stage vs 1-stage — Faster R-CNN · SSDLite	강의 + 실습	D4-5 Anchor 와 RPN
6	15:00–15:50	YOLO 구조 — grid · 출력 텐서 · 디코드	강의 + 실습	D4-6 YOLO grid 와 출력 텐서
7	16:00–16:50	영상 탐지 파이프라인	강의 + 실습	D4-7 프레임 파이프라인
8	17:00–17:50	미니 프로젝트 — 탐지기 2 종 평가 + 영상 인원 카운트	개인 실습	D4-8 탐지기 고르기 — threshold 와 해상도

오늘의 자료 · 온라인 링크

예측 (애니메이션) → 검증 (노트북) → 점검 (워크시트) — 모두 브라우저에서 바로 열립니다

기본 주소 (Day 4)

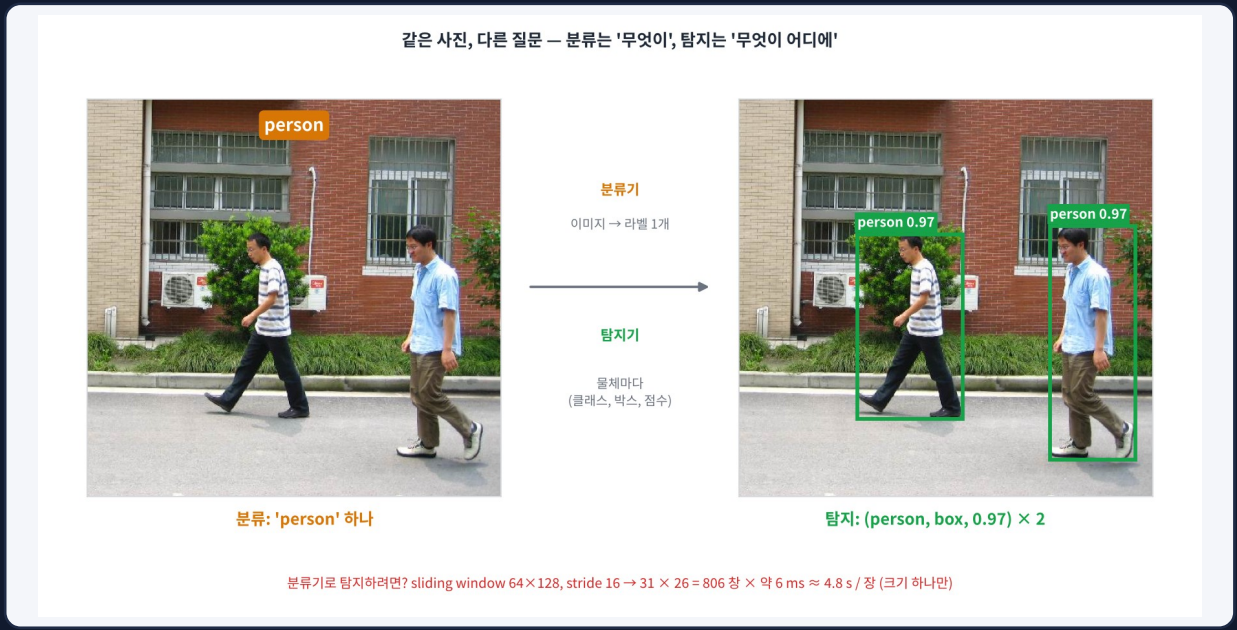
`https://imguru-mooc.github.io/AI_CAMPUS/VB-day4/`

animation.html	애니메이션 8 개 (D4-1 ~ D4-8) — #d4-3 처럼 바로가기
worksheet.html	개념 워크시트 — #t1 ~ #t8 교시 바로가기
VB_Day4_lecture.ipynb	강의 노트북 — 1~8 교시 문제 + 미니 프로젝트 (Colab · T4 GPU)
VB_Day4.pdf	오늘 슬라이드 PDF
data/people.mp4	보행자 영상 (OpenCV vtest) — 768×576 · 10 FPS · 795 프레임 · 7~8 교시
Day 1~3 복습 퀴즈	09:00 시작 — 강사가 Google Forms 링크 공유 (20 문항 · 15 분)

노트북은 " 파일 → Drive 에 사본 저장 " 후 작업 · 런타임 → 런타임 유형 변경 → T4 GPU · 10/30 특강으로 하루 공백 — 퀴즈로 Day 1~3 복습

오늘의 큰 그림 — 분류에서 탐지로

" 무엇이 " → " 무엇이 어디에 " — 물체마다 (클래스 , 박스 , 점수) · bbox → IoU → NMS → mAP → YOLO



오늘의 데이터 · 모델

- Penn-Fudan Pedestrian — 170 장 (Fudan 74 + Penn 96) · 보행자 423 명 · bbox + 마스크
- data/people.mp4 — 768×576 · 10 FPS · 795 프레임 (79.5 초)
- YOLOv8n / YOLOv8s (COCO 사전학습) — 3,157,200 / 11,166,560 params · stride 8 · 16 · 32
- person = YOLO 클래스 0 · torchvision 라벨 1 (0 = background)

torchvision 탐지기 (COCO 사전학습) — 5 교시는 아래 두 MobileNetV3-Large 모델을 비교

모델	방식	파라미터	GFLOPS	COCO box mAP
Faster R-CNN ResNet-50 FPN	2-stage	41,755,286	134.38	37.0
Faster R-CNN MobileNetV3-Large FPN	2-stage	19,386,354	4.49	32.8
SSDLite320 MobileNetV3-Large	1-stage	3,440,060	0.58	21.3

1 복습 퀴즈 · 분류기로 " 사람이 몇 명 ?"

09:00–09:50

학습 목표

- 09:00–09:15 Day 1~3 복습 퀴즈 (20 문항)
- 분류 = 이미지 한 장 → 라벨 하나
- 탐지 = 물체마다 (클래스 , 박스 , 점수)
- 창마다 분류기를 돌리는 sliding window 는 너무 느리다

노트북 실습

1. 문제 1-1 어노테이션 파싱
2. 문제 1-2 데이터셋 통계
3. 문제 1-3 sliding window 비용

▶ 애니메이션 D4-1 분류 vs 탐지

핵심 개념

1-1 파싱

```
Bounding box for object 1 "PASpersonWalking"
(Xmin, Ymin) - (Xmax, Ymax) : (160, 182) - (302, 431)
정규식 → FudanPed00001 = 2 명
[[160, 182, 302, 431], [420, 171, 535, 486]]
```

1-2 통계

```
Penn-Fudan 170 장 (Fudan 74 + Penn 96) · 보행자 423 명
이미지당 사람 수 분포 · bbox + 마스크
주의 : 작거나 많이 가려진 사람은 정답에 없다
```

1-3 sliding window

```
559×536 이미지 · 창 64×128 · stride 16 ( 크기 하나만 )
가로 (559−64)//16+1 = 31 · 세로 (536−128)//16+1 = 26
→ 806 창 × Day 3 MobileNetV3-S 약 6 ms ≈ 4.8 초 / 장
```

분류 vs 탐지

```
분류 이미지 → 라벨 하나 ('person')
탐지 물체마다 ( 클래스 , 박스 , 점수 ) → 사람 수 = 박스 수
```

체크포인트

창 크기 · 비율을 여러 개로 늘리면 더 느려진다 → 한 번의 forward 로 물체마다 클래스와 박스를 예측하는 탐지기가 필요하다 .

2 탐지 문제 정의 — bbox 표현 · COCO 포맷

10:00–10:50

학습 목표

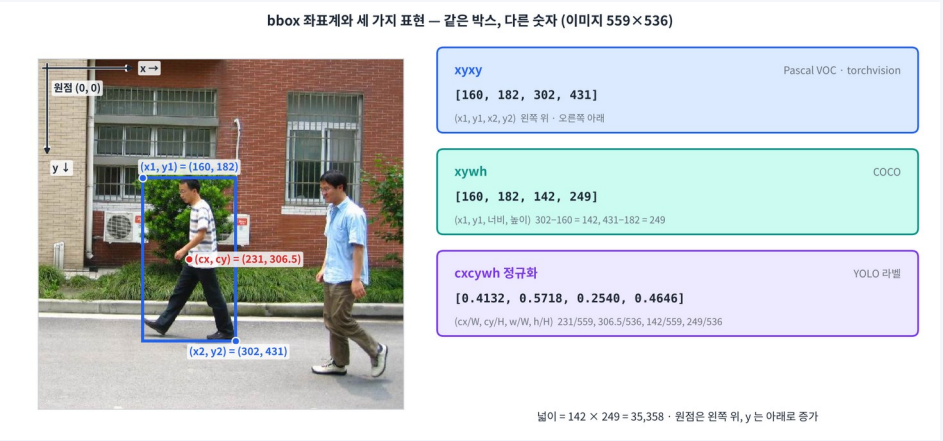
- 좌표계 : 원점 왼쪽 위 · x 오른쪽 · y 아래
- xyxy — Pascal VOC · torchvision
- xywh (왼쪽 위 x, y, 너비 , 높이) — COCO
- cxcywh 정규화 (0~1) — YOLO 라벨

노트북 실습

1. 문제 2-1 좌표 변환
2. 문제 2-2 정답 박스 그리기
3. 문제 2-3 COCO JSON 만들기

▶ 애니메이션 D4-2 bbox 좌표계

핵심 개념



2-3 COCO

```
images       id, file_name, width, height
annotations  id, image_id, category_id, bbox=xywh, area, iscrowd
categories   [{id: 1, name: "person"}]  → annotations 423 개
```

손실

```
탐지 손실 = 분류 손실 (cross-entropy)
          + 박스 회귀 손실 (Smooth L1 / IoU 계열 )
```

체크포인트

형식을 섞으면 박스가 엉뚱한 곳에 그려진다 . 데이터를 받으면 먼저 어떤 형식인지 , 정규화됐는지 확인하고 2-2 처럼 직접 그려 본다 .

3 IoU · NMS

11:00–11:50

학습 목표

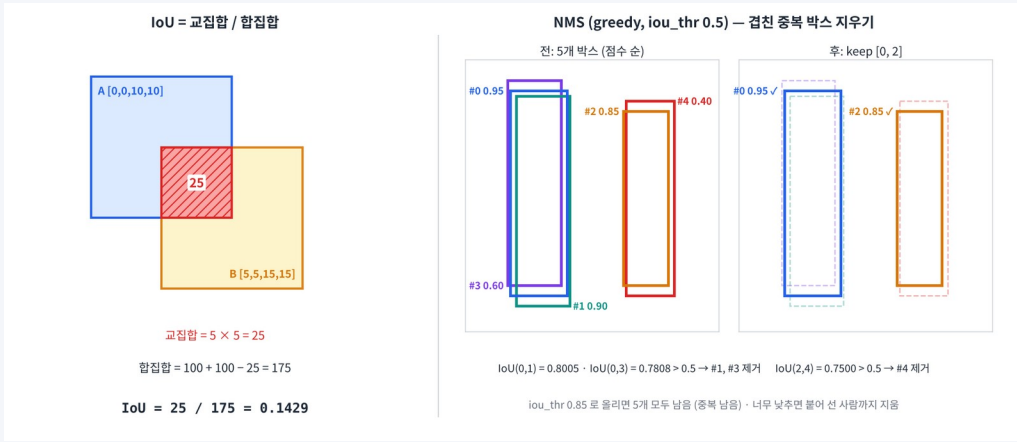
- IoU = 교집합 / 합집합 (0~1)
- 교집합 폭 = max(0, min(x2) − max(x1)), 높이도 같이
- NMS (greedy) — 겹친 중복 박스 지우기
- conf threshold vs IoU threshold

노트북 실습

1. 문제 3-1 iou
2. 문제 3-2 iou_matrix vs box_iou
3. 문제 3-3 nms vs torchvision.ops.nms
4. 문제 3-4 threshold 실험

▶ 애니메이션 D4-3 IoU · NMS

핵심 개념



3-1 · 3-2

```
[50,50,150,150] vs [60,60,160,160] → 8,100 / 11,900 = 0.6807
같은 박스 1 · 안 겹치면 0
iou_matrix(A, B) (N×M, 브로드캐스팅) = torchvision.ops.box_iou
```

3-3 · 3-4

```
점수 순 정렬 → 최고 keep → IoU > iou_thr 제거 → 반복
iou_thr 0.5 → keep [0, 2] · iou_thr 0.85 → 5개 모두 남음
conf_thr 0.5 먼저 → 4 빠지고 NMS 후 [0, 2]
```

체크포인트

conf threshold = 점수 낮은 박스 버리기 (탐지 개수 · 오탐 조절). IoU threshold 가 낮으면 붙어 있는 사람까지 지우고, 높으면 중복이 남는다.

4 평가 지표 — Precision · Recall · AP · mAP

13:00–13:50

학습 목표

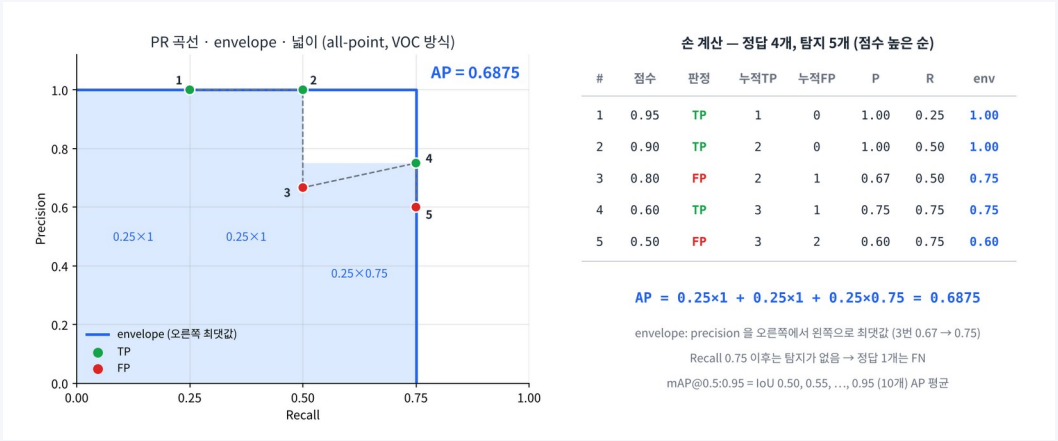
- TP = 정답과 IoU ≥ 0.5 (처음 매칭만)
- Precision = TP/(TP+FP) · Recall = TP/ 정답 수
- AP = envelope 아래 넓이 (all-point, VOC)
- mAP@0.5:0.95 = IoU 10 개 AP 평균 (COCO)

노트북 실습

1. 문제 4-1 precision_recall
2. 문제 4-2 ap_from_pr
3. 문제 4-3 match_detections
4. 문제 4-4 evaluate_ap · map_50_95

▶ 애니메이션 D4-4 PR 곡선과 AP

핵심 개념



#	점수	판정	누적TP	누적FP	P	R	env
1	0.95	TP	1	0	1.00	0.25	1.00
2	0.90	TP	2	0	1.00	0.50	1.00
3	0.80	FP	2	1	0.67	0.50	0.75
4	0.60	TP	3	1	0.75	0.75	0.75
5	0.50	FP	3	2	0.60	0.75	0.60

4-3 매칭

모든 이미지의 탐지를 점수 높은 순 정렬
IoU ≥ 0.5 + 같은 정답에 처음 매칭 → TP
나머지 (중복 포함) → FP · 못 찾은 정답 → FN

4-4 mAP

mAP@0.5:0.95 = IoU 0.50, 0.55, …, 0.95 (10 개) AP 평균
오늘은 person 하나 → mAP = AP
함수들 → iou_nms_map.py 로 저장

체크포인트

COCO 는 101 점 보간이라 값이 약간 다를 수 있다 . mAP@0.5:0.95 는 박스 위치 정확도까지 본다 — 8 교시에서 직접 만든 evaluate_ap 로 YOLOv8 을 평가한다 .

5 2-stage vs 1-stage — Faster R-CNN · SSDLite

14:00–14:50

학습 목표

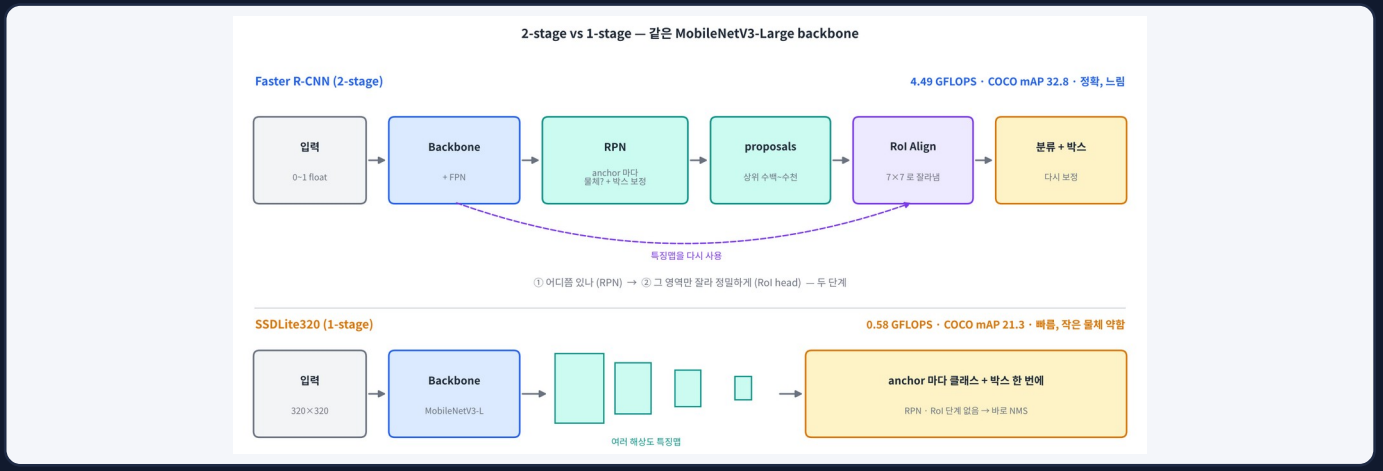
- Anchor = 특징맵 위치마다 크기 · 비율이 다른 기준 박스
- 모델은 오프셋 (dx, dy, dw, dh) + 점수 예측
- 2-stage: RPN → proposal → RoI Align → 분류
- 1-stage: 여러 특징맵에서 한 번에

노트북 실습

1. 문제 5-1 anchor 만들기
2. 문제 5-2 Faster R-CNN vs SSDLite 추론
3. 문제 5-3 인원 수 비교

▶ 애니메이션 D4-5 Anchor 와 RPN

핵심 개념



5-1 anchor

```
559×536 이미지 · stride 16 → 특징맵 34×35 = 1,190 위치
× ( 크기 3 × 비율 3 = 9) = 10,710 anchor
```

5-2 입출력

```
입력 : 0~1 float 텐서 리스트 ( 정규화 · 크기 조정은 모델 안 )
출력 dict: "boxes" (N,4) xyxy · "labels" (N,) · "scores" (N,)
person = 라벨 1 (0 = background)
```

체크포인트

2-stage 는 proposal 영역을 잘라 다시 보기 때문에 정확하지만 느리다 . 1-stage 는 한 번에 예측해 빠르지만 작은 물체에 약하다 — 5-3 에서 두 탐지기의 인원 수를 비교한다 .

6 YOLO 구조 — grid · 출력 텐서 · 디코드

15:00–15:50

학습 목표

- Backbone (CSP) → Neck (FPN + PAN) → Head (stride 8 · 16 · 32)
- YOLOv8 = anchor-free (DFL) · 클래스 점수 sigmoid
- YOLOv8n 3,157,200 · YOLOv8s 11,166,560 params

노트북 실습

1. 문제 6-1 예측 개수
2. 문제 6-2 letterbox · 원시 출력
3. 문제 6-3 decode_yolo vs ultralytics
4. 문제 6-4 YOLOv8n vs s

▶ 애니메이션 D4-6 YOLO grid 와 출력 텐서

핵심 개념



6-2 전처리

```
letterbox: scale = min(640/H, 640/W) + 회색(114) 패딩
BGR→RGB · /255 · CHW → model.model(x)[0] = (1, 84, 8400)
```

6-3 디코드

```
(8400, 84) 전치 → 클래스 최댓값 = 점수 → conf ≥ 0.25
→ cxcywh → xyxy → NMS (IoU 0.7) → 패딩 빼고 ÷ scale
= ultralytics model.predict 결과 (r.boxes.xyxy · conf · cls)
```

체크포인트

YOLOv1 (2016, grid) → v2/v3 (anchor · 다중 스케일) → v4/v5 (CSP · 증강) → v8 (2023, anchor-free · decoupled head). 커스텀 학습은 YOLOv8 Edge 과목 (12/11~).

7 영상 탐지 파이프라인

16:00–16:50

학습 목표

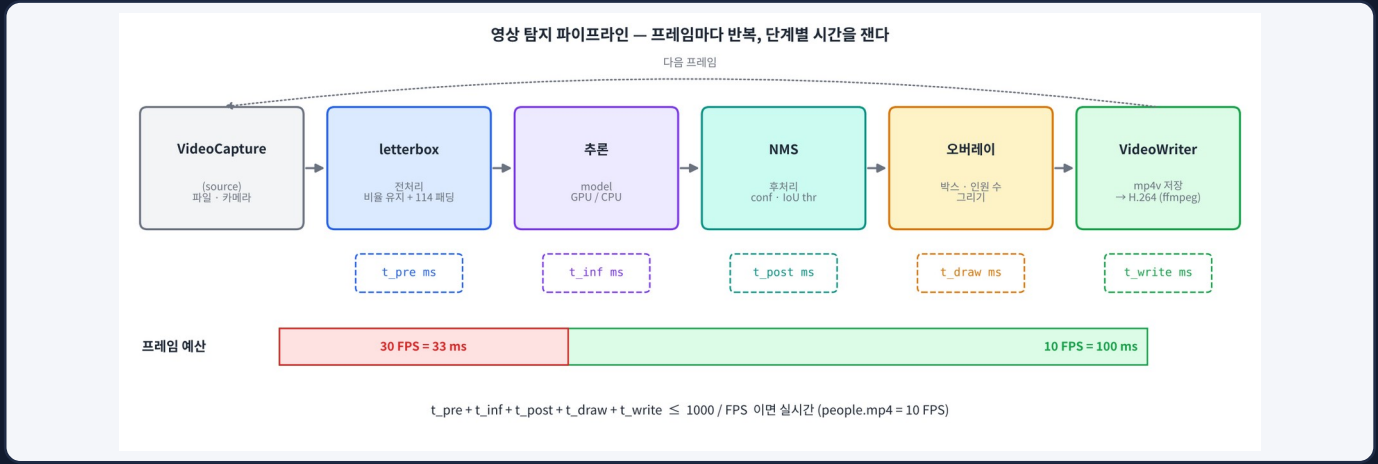
- cv2.VideoCapture(source) — source 하나로, 나중에 카메라로 바꾸기 쉽게
- 전처리 → 추론 → NMS → 오버레이 → 저장 (mp4v) → H.264 (ffmpeg)
- 단계별 시간 측정 · GPU vs CPU

노트북 실습

1. 문제 7-1 detect_people
2. 문제 7-2 process_video
3. 문제 7-3 단계별 시간

▶ 애니메이션 D4-7 프레임 파이프라인

핵심 개념



영상

```
data/people.mp4 — OpenCV 공개 샘플 (vtest) 보행자
768×576 · 10 FPS · 795 프레임 (79.5 초)
```

7-3 (실행 예)

```
CPU · YOLOv8n 640
전처리 2.8 ms · 추론 102 ms · 후처리 1.3 ms
→ 전체 약 9.0 FPS
```

체크포인트

실시간 조건 : 10 FPS 영상 → 프레임당 100 ms, 30 FPS → 33 ms. 추론만이 아니라 전처리 · 그리기 · 저장까지 합친 시간으로 판단한다.

8 미니 프로젝트 — 탐지기 2 종 평가 + 영상 인원 카운트

17:00–17:50

직접 만든 mAP (iou_nms_map.py) 로 YOLOv8n · YOLOv8s 평가 → 운영 모델 · threshold · 해상도 고르기

과제	내용	배점
1	직접 만든 evaluate_ap 로 YOLOv8n · YOLOv8s 를 Penn-Fudan 170 장에서 평가 (640, conf 0.001) — AP@0.5 · mAP@0.5:0.95	20
2	입력 해상도 320 / 640 × 모델 2 종 → AP · mAP · ms/ 장 표	20
3	conf threshold 0.1~0.9 에서 Precision · Recall · F1 → 운영 threshold 고르기	20
4	영상 인원 카운트 — 고른 모델 · threshold 로 프레임마다 사람 수 , 결과 영상 + CSV	20
5	결과 해석 (강사 채점)	20

채점 셀 실행 → 자동 채점 80 점 (과제 1~4) + 강사 20 점 (과제 5)

실행 예 (강사 사전 실행 , CPU)

모델 · 입력	AP@0.5	mAP@0.5:0.95	ms/ 장
YOLOv8n · 640	0.9692	0.8357	121
YOLOv8s · 640	0.9663	0.8498	298
YOLOv8n · 320	0.9741	0.8159	37.7
YOLOv8s · 320	0.9674	0.8381	98.8

threshold F1 최대 conf 0.70 (P 0.8923, R 0.9598, F1 0.9248)
영상 795 프레임 , 최대 8 명 , 평균 5.0 명

해석 포인트

크기 Penn-Fudan 처럼 사람이 크면 작은 모델 · 낮은 해상도로도 AP50 이 거의 같다
고르기 → mAP@0.5:0.95 (위치 정밀도) 와 속도로 고른다
conf 낮추면 Recall↑ Precision↓
FP 정답에 없는 사람 (작거나 가려진) 이 FP 로 잡힌다

▶ 애니메이션 D4-8 탐지기 고르기 — threshold 와 해상도

오늘의 핵심 정리

각 항목 옆 번호는 다시 볼 애니메이션

분류 vs 탐지	탐지 = 물체마다 (클래스 , 박스 , 점수) · sliding window 806 창 ≈ 4.8 초 / 장	D4-1
bbox · COCO	xyxy (VOC · torchvision) · xywh (COCO) · cxcywh 정규화 (YOLO) · COCO JSON	D4-2
IoU · NMS	IoU = 교집합 / 합집합 · greedy NMS · 예제 keep [0, 2] · conf vs IoU threshold	D4-3
AP · mAP	P = TP/(TP+FP) · R = TP/ 정답 · envelope 넓이 AP 0.6875 · mAP@0.5:0.95	D4-4
2-stage · 1-stage	anchor 10,710 개 · Faster R-CNN (RPN · RoI Align) vs SSDLite (한 번에)	D4-5
YOLOv8	Backbone → Neck → Head · 8,400 예측 · (1, 84, 8400) · letterbox → 디코드 → NMS	D4-6
영상 파이프라인	VideoCapture → 전처리 → 추론 → NMS → 오버레이 → 저장 · 10 FPS = 100 ms	D4-7
미니 프로젝트	직접 만든 mAP 로 YOLOv8n · s × 320 / 640 평가 · threshold · 영상 인원 카운트	D4-8

Day 5 로 가는 다리 — 세그 · 포즈 · ONNX · 성능 측정

박스 다음은 픽셀과 관절 , 그리고 배포 — 마지막 날에는 위험구역 침입 감지를 만듭니다

Day 4 (오늘)		Day 5 (11/3 화)
탐지 : 물체마다 bbox	→	세그멘테이션 : 픽셀마다 마스크
사람 박스 · 인원 수	→	포즈 : 사람마다 관절 keypoint
ultralytics · PyTorch 추론	→	ONNX 로 내보내 추론
단계별 시간 · r.speed	→	성능 측정 — 지연시간 · FPS
영상 인원 카운트	→	위험구역 침입 감지
iou_nms_map.py · 영상 파이프라인	→	**그대로 사용**

세그 · 포즈 · ONNX · 성능 측정 → 위험구역 침입 감지

이후 : ONNX · TensorRT (11/4~) → Quantization · Pruning (11/16~) → YOLOv8 Edge 모델 (12/11~, 커스텀 학습)